

LeafGuard AI — Plant Disease Detection System

Phase 1 Report: Dataset Selection & Data Preprocessing | Lead Developer: Tayyab Sohail

1. Problem Statement

In modern agriculture, early detection of crop leaf diseases is vital to prevent crop failure and maximize yield. Traditional disease diagnosis requires specialized agronomists and lab equipment, which is inaccessible to many smallholder farmers. LeafGuard AI addresses this by providing an automated computer-vision tool that analyzes leaf photographs, detects pathologies with high confidence, offers Grad-CAM visual justifications, and provides actionable treatment recommendations.

2. Dataset Selection & Justification

To build a robust classification engine, we utilize the publicly available **PlantVillage Dataset** as our primary training resource. It contains 54,305 high-quality leaf images across 14 crop species and 38 classes (both healthy and diseased). To mitigate the laboratory-controlled background bias of PlantVillage, we validated our data pipeline against the **PlantDoc Dataset** (2,598 field-captured leaf images under natural illumination). This combination ensures that the model retains high accuracy in ideal conditions and is highly generalized for real-world application in low-resource settings.

3. Dataset Preprocessing Pipeline

A dual-mode preprocessing pipeline was executed to prepare both the dataset metadata (tabular features) and raw images:

Preprocessing Step	Description	Justification
Duplicate & Missing Handling	Identified and dropped 50 duplicate records. Imputed missing file sizes & brightness metrics using the column median; imputed missing crop types with the mode.	Ensures data integrity and prevents bias/data leakage in evaluation partitions.
Outlier Capping (IQR)	Clipped extreme values in image properties (file size, brightness, blurriness) using IQR [Q1 - 1.5*IQR, Q3 + 1.5*IQR].	Prevents highly corrupted, overexposed, or extremely blurry uploads from distorting downstream statistics.
Encoding & Scaling	Encoded the target label ('status') to binary (1=Healthy, 0=Diseased). Standardized numerical features using StandardScaler.	Packs features into standard distributions (mean=0, std=1) for stable gradient descent in classification heads.
Class Balancing (SMOTE-Tomek)	Applied SMOTE-Tomek to the tabular metadata, resolving class imbalance (898 Healthy vs 1102 Diseased) to achieve perfect 1047:1047 balance.	Eliminates majority-class bias. Raw images use pixel-level augmentations, while metadata balances via interpolation.
Image Pipeline (CV)	Resized raw leaf images to 256x256 pixels, normalized pixel intensities to [0,1], and applied random rotations, zoom, and flips.	Fits base network requirements, reduces compute footprint, and prevents model overfitting to static orientations.

4. Technical Stack & Libraries

Programming Environment: Python 3.12, Jupyter Notebook, VS Code, Git/GitHub.

Libraries Used: *Pandas* & *NumPy* (data manipulation & array operations), *Scikit-learn* (scaling, encoding, split evaluation), *Imbalanced-learn* (SMOTE-Tomek implementation), *Matplotlib* & *Seaborn* (exploratory plotting and data distribution profiling).

5. Challenges & Solutions

Tabular vs Image Resampling: We resolved the challenge of applying class balancing. Raw image data cannot be resampled via SMOTE due to non-manifold pixel interpolations. Our solution implements pixel data augmentation (flips, rotations) for the image classifier while utilizing SMOTE-Tomek exclusively on structural metadata features.

Outlier Identification: Highly underexposed/blurry leaf photos distort statistical profiling. We established a threshold boundary using IQR on image brightness/blurriness which will double as the real-time quality assurance filter (`detectImageQuality()`) in the production pipeline.